

Ciencias de la Computación II

Informe de laboratorio

Tema: Comparación de los algoritmos de Recubrimiento Mínimo Prim y Kruskal

Profesor: Simar Herrera

Integrantes:

Sebastián Rangel - 20231020006

Miguel Bueno - 20251020093

Sebastián Zambrano - 20251020102

Diego Yañes - 20251020103

Intersemestrales Junio - Julio del 2026
Universidad Distrital Francisco José de
Caldas

Introducción

Durante el desarrollo del curso se estudiaron los algoritmos de recubrimiento mínimo Prim y Kruskal, los cuales permiten obtener un Árbol de Recubrimiento Mínimo (Minimum Spanning Tree, MST) de un grafo no dirigido y conexo.

Ambos algoritmos tienen el mismo objetivo: conectar todos los vértices del grafo utilizando el menor peso posible y evitando la formación de ciclos. Sin embargo, cada uno emplea una estrategia diferente para construir el árbol mínimo.

En este laboratorio se implementaron ambos algoritmos utilizando la representación de grafos desarrollada previamente en Java. Posteriormente se realizaron diferentes casos de prueba con el fin de comparar su comportamiento, medir el tiempo de ejecución e identificar en qué situaciones uno de ellos presenta un mejor desempeño.

Algoritmo de Prim

El algoritmo de Prim construye el árbol de recubrimiento mínimo comenzando desde un vértice cualquiera del grafo.

En cada iteración selecciona la arista de menor peso que conecta un vértice que ya pertenece al árbol con otro que aún no ha sido visitado. El proceso continúa hasta incluir todos los vértices del grafo.

Esta estrategia permite que el árbol vaya creciendo progresivamente hasta conectar todos los nodos.

Su implementación resulta especialmente eficiente cuando el grafo posee una gran cantidad de aristas.

Complejidad:

" $O(E \log V)$ " utilizando una cola de prioridad.

Algoritmo de Kruskal

El algoritmo de Kruskal utiliza una estrategia diferente.

En lugar de comenzar desde un vértice específico, ordena todas las aristas del grafo de menor a mayor peso y posteriormente las va agregando al árbol únicamente cuando no producen ciclos.

Para detectar dichos ciclos se utiliza normalmente la estructura Union-Find o conjuntos disjuntos.

Este algoritmo resulta muy eficiente cuando el número de aristas es reducido.

Complejidad:

" $O(E \log V)$ ", considerando el proceso de ordenamiento de las aristas.

Casos de prueba

Con el fin de comparar ambos algoritmos, se realizaron diferentes pruebas utilizando grafos con distintas características.

Caso de prueba 1

Grafo pequeño.

- 5 vértices.
- 7 aristas.

Resultado esperado:

Ambos algoritmos generan exactamente el mismo árbol de recubrimiento mínimo y el tiempo de ejecución es prácticamente igual debido al pequeño tamaño del grafo.

Caso de prueba 2

Grafo mediano.

- 20 vértices.
- 35 aristas.

Resultado esperado:

Los dos algoritmos encuentran el mismo costo total del árbol mínimo.

Prim presenta una ligera ventaja en el tiempo de ejecución.

Caso de prueba 3

Grafo denso.

- 100 vértices.
- 450 aristas.

Resultado esperado:

Prim mantiene un mejor rendimiento debido a que trabaja directamente sobre los vértices y evita ordenar todas las aristas del grafo.

Kruskal requiere ordenar una cantidad considerable de aristas antes de iniciar la construcción del árbol.

Caso de prueba 4

Grafo disperso.

- 100 vértices.
- 120 aristas.

Resultado esperado:

Kruskal obtiene mejores tiempos de ejecución, ya que el número de aristas a ordenar es relativamente pequeño.

En este tipo de grafos ambos algoritmos continúan generando el mismo árbol mínimo.

Comparación de complejidad

Algoritmo	Complejidad
-----------	-------------

Prim	$O(E \log V)$
------	---------------

Kruskal	$O(E \log V)$
---------	---------------

Aunque ambos poseen una complejidad similar, el comportamiento práctico depende principalmente de la cantidad de aristas presentes en el grafo.

Comparación de comportamiento

Durante las pruebas realizadas se observó que ambos algoritmos generan exactamente el mismo árbol de recubrimiento mínimo, ya que resuelven el mismo problema utilizando estrategias diferentes.

La principal diferencia se encuentra en la forma como construyen el árbol.

Prim comienza desde un vértice y expande el árbol poco a poco seleccionando siempre la arista de menor costo disponible.

Por otra parte, Kruskal analiza todas las aristas del grafo, las ordena por peso y posteriormente las agrega evitando la formación de ciclos.

Estas diferencias hacen que cada algoritmo sea más conveniente dependiendo del tipo de grafo utilizado.

Casos donde ambos algoritmos presentan un comportamiento similar

Los algoritmos presentan un comportamiento muy parecido cuando:

- El número de vértices es pequeño.
- Existen pocas aristas.
- Los pesos de las aristas son similares.
- El grafo no posee una gran complejidad.

En estas condiciones la diferencia en el tiempo de ejecución es prácticamente despreciable.

Casos donde el comportamiento es diferente

Las diferencias comienzan a observarse cuando aumenta considerablemente el tamaño del grafo.

En grafos densos, donde existe una gran cantidad de conexiones, Prim suele obtener mejores tiempos de ejecución.

En cambio, cuando el grafo es disperso y posee pocas aristas, Kruskal generalmente presenta un mejor desempeño debido a que el proceso de ordenamiento resulta menos costoso.

Conclusiones

La implementación realizada permitió comprobar que tanto Prim como Kruskal generan el mismo árbol de recubrimiento mínimo para un mismo grafo.

Las diferencias entre ambos algoritmos no se encuentran en el resultado obtenido, sino en la estrategia utilizada para construir dicho árbol.

Durante las pruebas se observó que Prim presenta un mejor comportamiento en grafos con una gran cantidad de aristas, mientras que Kruskal resulta más conveniente cuando el grafo es disperso.

Finalmente, puede concluirse que ninguno de los dos algoritmos es mejor en todos los casos, ya que la elección depende de las características del problema y de la estructura del grafo que se desea analizar.